ARL Debugging Challenge

Rover Safety Monitor — Debug Report

Name: Mohamed Ahmed Fouad

Project: ARL-2027-Public-Mission / Debugging_Assignment

Summary

The supplied Rover Safety Monitor project failed to build and, once building, produced incorrect results due to a combination of build-configuration, coordinate-transform, unit-conversion, and logic errors. This report documents every issue found during debugging, its symptom, root cause, fix, and the underlying programming or engineering concept involved, followed by a walkthrough of how the corrected system processes a detection end-to-end, and a list of resources used.

After all fixes were applied, the program was rebuilt and run against the supplied telemetry in main.cpp and produced output matching the specification exactly:

```
ARL Rover Safety Monitor
Valid detections: 3
Nearest obstacle: cone
World position: (96.0, 62.0)
Stopping distance: 20.0 m
Emergency brake: ENGAGE
```

Issue 1: Missing Source File in CMakeLists.txt (safety_monitor.cpp not built)

Symptom

The project did not build and run correctly against the full implementation: functions declared in safety_monitor.hpp (processDetections, findNearestObstacle, calculateStoppingDistance, shouldEmergencyBrake) were not being compiled into the executable at all, because only main.cpp was listed as a source file for the target.

Root Cause

add_executable(rover_safety_monitor ...) defines exactly which .cpp files CMake compiles and links into the executable. src/safety_monitor.cpp — which contains the actual function definitions for everything declared in safety_monitor.hpp — was missing from that source list, so the header's declarations had no corresponding compiled implementation linked into the build.

Before:

```
add_executable(rover_safety_monitor
    main.cpp
)
```

After (Fix):

```
add_executable(rover_safety_monitor
    main.cpp
    src/safety_monitor.cpp
)
```

Reasoning

Adding src/safety_monitor.cpp to the add_executable() source list tells CMake to compile that translation unit and link its object file into rover_safety_monitor, so the implementations of processDetections, findNearestObstacle, calculateStoppingDistance, and shouldEmergencyBrake are actually present in the final executable rather than just declared in the header.

Programming Concept

CMake target/source-list configuration: an executable target only contains the source files explicitly listed for it (via add_executable or target_sources). Including a header (#include "safety_monitor.hpp") only brings in declarations at compile time — the corresponding .cpp file providing the actual definitions must separately be added to the target, or those symbols are never built and the project cannot correctly link or behave as specified.

Issue 2: Inverted Comparison in findNearestObstacle

Symptom

The function intended to find the closest obstacle instead returned the farthest one.

Root Cause

The loop updated the running 'nearest' pointer whenever it found an obstacle with a larger range than the current best, which selects the maximum, not the minimum.

Before:

```
const Obstacle* nearest = &obstacles.front();
for (const auto& obstacle : obstacles) {
    if (obstacle.range > nearest->range) {
        nearest = &obstacle;
    }
}
```

After (Fix):

```
const Obstacle* nearest = &obstacles.front();
for (const auto& obstacle : obstacles) {
    if (obstacle.range < nearest->range) {
        nearest = &obstacle;
    }
}
```

Reasoning

Flipping '>' to '<' makes the loop retain the obstacle with the smallest range seen so far, which is the correct definition of 'nearest'.

Programming Concept

Selection/reduction algorithms (finding a min or max via a single linear pass) depend entirely on comparison direction; a single flipped operator silently inverts the algorithm's meaning without any compiler warning.

Issue 3: Off-by-One Loop Bound in processDetections

Symptom

One fewer obstacle than expected was reported as valid; the last detection in the telemetry list was silently dropped from processing.

Root Cause

The loop condition was 'index + 1 < detections.size()', which stops one index short of the final element, so the last Detection in the vector was never examined.

Before:

```
for (std::size_t index = 0; index + 1 < detections.size(); ++index) {
```

After (Fix):

```
for (std::size_t index = 0; index < detections.size(); ++index) {
```

Reasoning

Changing the bound to 'index < detections.size()' allows the loop to visit every valid index in the vector, including the last one.

Programming Concept

Classic off-by-one error in loop bounds; with zero-based indexing and size() returning a count (not a max index), the correct half-open range is [0, size()).

Issue 4: Inverted Confidence Validity Check

Symptom

Detections with low confidence were accepted as valid, while genuinely reliable, high-confidence detections were rejected — the opposite of the intended filter.

Root Cause

The condition checked 'confidence <= config.minimumConfidence', which admits low-confidence readings and rejects everything at or above the configured minimum.

Before:

```
const bool validConfidence = detection.confidence >= 0.0
    && detection.confidence <= config.minimumConfidence;
```

After (Fix):

```
const bool validConfidence = detection.confidence <= 1.0
    && detection.confidence >= config.minimumConfidence;
```

Reasoning

Per the specification, a detection is valid when its confidence lies between the configured minimum and 1.0, inclusive. The corrected condition requires confidence to be at least the configured minimum and at most the maximum possible value (1.0).

Programming Concept

Boundary/range validation logic: care must be taken that comparison operators express the intended inequality (>= lower bound, <= upper bound), especially when a threshold value could easily be mistaken for either bound.

Issue 5: Missing Degrees-to-Radians Conversion for Heading

Symptom

World-frame coordinates for obstacles were computed incorrectly for any heading other than very small angles; cos/sin of the heading did not behave as expected.

Root Cause

std::cos and std::sin expect arguments in radians, but the code assigned pose.headingDegrees directly to headingRadians with no conversion, effectively treating a degree value as if it were already in radians.

Before:

```
const double headingRadians = pose.headingDegrees;
```

After (Fix):

```
constexpr double kPi = 3.14159265358979323846;
const double headingRadians = pose.headingDegrees * kPi / 180.0;
```

Reasoning

The standard conversion radians = degrees * (pi / 180) was applied. A full-precision pi constant was used (rather than a short literal like 3.14159) so the conversion is accurate at every heading, not only ones where a coarse approximation happens to be close enough.

Programming Concept

Unit conversion between angular measurement systems (degrees vs. radians) is a common source of silent numerical errors in graphics, robotics, and navigation code, because the code compiles and runs without any error — it just produces a plausible-looking but wrong number.

Issue 6: Sign Error in World-Frame X Coordinate Transform

Symptom

Computed world coordinates for obstacles did not match the expected values from the specification example (expected (96.0, 62.0) for the 'cone' obstacle; the uncorrected formula produced a different X coordinate).

Root Cause

The local-to-world rotation added the sine term for both X and Y: worldX = poseX + cos(heading)*forward + sin(heading)*left. This is incorrect — rotating the local (forward, left) frame into the world frame requires the X component of the 'left' direction vector to be negative sine, not positive.

Before:

```
pose.worldX + cosine * detection.forward + sine * detection.left,
pose.worldY + sine * detection.forward + cosine * detection.left,
```

After (Fix):

```
pose.worldX + cosine * detection.forward - sine * detection.left,
pose.worldY + sine * detection.forward + cosine * detection.left,
```

Reasoning

In the rover's local frame, 'forward' and 'left' are two perpendicular directions. Rotated into the world frame by heading angle θ, the forward direction vector is (cosθ, sinθ) and the left direction vector (90° counter-clockwise from forward) is (-sinθ, cosθ). An obstacle's world position is the rover's position plus forward·(forward vector) plus left·(left vector), giving:

worldX = poseX + forward·cosθ − left·sinθ worldY = poseY + forward·sinθ + left·cosθ

Verification with the supplied example (forward=12, left=4, heading=90°, pose=(100,50)): cos90°=0, sin90°=1, so worldX = 100 + 12(0) − 4(1) = 96, worldY = 50 + 12(1) + 4(0) = 62 — matching the specification's expected (96.0, 62.0) exactly.

Programming Concept

2D coordinate frame transformation via rotation matrices. Converting a point from one rotated reference frame (robot-local) to another (world/global) is a standard robotics operation; a sign error in a rotation term produces plausible-looking but geometrically wrong output that is easy to miss without a known-good numeric example to check against.

Issue 7: Missing / Incorrect Speed Unit Conversion (km/h to m/s)

Symptom

Calculated stopping distance was consistently too large (e.g. producing 165.6 m for input that should give 20.0 m per the specification example).

Root Cause

calculateStoppingDistance received speedKph (kilometres per hour) but used it directly as speedMps (metres per second) with no conversion. A later fix attempt ('speedKph / (18/5)') also failed silently because 18 and 5 are int literals, so 18/5 evaluates via integer division to 3 (not 3.6) before ever becoming a double.

Before:

```
const double speedMps = speedKph;
```

After (Fix):

```
const double speedMps = speedKph / 3.6;
```

Reasoning

1 km/h = 1000 m / 3600 s = 1/3.6 m/s, so dividing by 3.6 converts km/h to m/s correctly. Verified against the specification: 36 km/h / 3.6 = 10 m/s; reactionDistance = 10 * 1.0 = 10 m; brakingDistance = 10^2 / (2*5) = 10 m; total = 20.0 m, matching the expected output exactly.

Programming Concept

Unit consistency in physical formulas.

Issue 8: Emergency Brake Logic Was Unconditionally False / Unimplemented

Symptom

The program always printed 'Emergency brake: clear', regardless of how close any obstacle was to the rover.

Root Cause

shouldEmergencyBrake was an empty stub that unconditionally returned false and never examined the obstacles, speed, or configuration at all.

Before:

```
bool shouldEmergencyBrake(const std::vector<Obstacle>& obstacles, double speedKph,
                          const SafetyConfig& config) {
    return false;
}
```

After (Fix):

```
bool shouldEmergencyBrake(const std::vector<Obstacle>& obstacles, double speedKph,
                          const SafetyConfig& config) {
    const double stoppingDistance = calculateStoppingDistance(speedKph, config);
    for (const auto& obstacle : obstacles) {
        const bool inFront = obstacle.forward >= 0.0;
        const bool inLane = std::fabs(obstacle.left) <= config.laneHalfWidthMeters;
        const bool withinStoppingRange = obstacle.forward <= stoppingDistance;
        if (inFront && inLane && withinStoppingRange) {
            return true;
        }
    }
    return false;
}
```

Reasoning

Per the specification, braking must engage if any valid obstacle is simultaneously: (1) in front of the rover, forward ≥ 0 (inclusive); (2) inside the lane corridor, |left| ≤ lane_half_width (boundary inclusive); and (3) within stopping distance, forward ≤ stoppingDistance (inclusive). All three conditions must be checked per-obstacle, over the full obstacle list — not only against the single nearest obstacle, since the nearest obstacle by straight-line range is not necessarily the one directly in the rover's path. In the supplied example, 'cone' is nearest overall (range ≈ 12.65 m) but lies outside the lane corridor (|left|=4.0 > 1.50), while 'barrier' is farther by range but sits inside the lane and within stopping distance (forward=18.0 ≤ 20.0), so it is the one that correctly triggers ENGAGE.

Programming Concept

Existential quantification over a collection ('does at least one element satisfy all of these conditions') implemented as a linear scan with early-return on the first match; and the importance of checking the specification's exact predicate rather than substituting a simpler-but-different one (e.g. 'nearest obstacle' vs. 'any obstacle meeting the criteria').

Issue 8b: Reversed Comparison in First shouldEmergencyBrake Attempt

A first working attempt at the braking loop wrote 'calculateStoppingDistance(...) <= obstacle.forward', which reads as “stopping distance is less than or equal to the obstacle's forward distance” — i.e. the obstacle is farther away than the stopping distance, the opposite of when braking is needed. Checked against the sample data (barrier: forward=18, stoppingDistance=20), this evaluated 20 ≤ 18 → false, so 'barrier' would not trigger braking, contradicting the specification's expected ENGAGE result. The fix was to compare in the correct order: obstacle.forward ≤ stoppingDistance.

End-to-End Walkthrough: Detection to Braking Decision

Using the supplied telemetry (pose = (100.0, 50.0), heading = 90°; config: minimumConfidence=0.60, maximumRangeMeters=40.0, laneHalfWidthMeters=1.50, reactionTimeSeconds=1.0, maximumDecelerationMps2=5.0; speed=36 km/h), the corrected system processes the detection list as follows:

1. Finiteness check (isFinite)

Every Detection's forward, left, and confidence fields are checked with std::isfinite to reject NaN or infinite sensor readings before any further processing.

2. Range computation

For each detection, range = std::hypot(forward, left) is computed — the straight-line distance from the rover to the detected point in the local frame.

3. Validity filtering (processDetections)

A detection becomes an Obstacle only if: all fields are finite, confidence is within [minimumConfidence, 1.0], and range is within (0, maximumRangeMeters]. Applied to the sample data: 'barrier' (conf 0.90, range≈18.03) and 'crate' (conf 0.75, range≈25.00) and 'cone' (conf 0.95, range≈12.65) all pass; 'sign' fails (confidence 0.55 < 0.60); 'debris' fails (range 45 > 40). Result: 3 valid obstacles.

4. Local-to-world coordinate transform

For each valid obstacle, heading is converted from degrees to radians, and the rotation worldX = poseX + forward·cosθ − left·sinθ, worldY = poseY + forward·sinθ + left·cosθ maps the local (forward, left) reading into shared world coordinates.

5. Nearest-obstacle selection (findNearestObstacle)

A single linear scan over the valid obstacles keeps a pointer to whichever has the smallest range seen so far. For the sample data, 'cone' (range≈12.65) is nearest, with world position (96.0, 62.0).

6. Stopping distance calculation (calculateStoppingDistance)

Speed is converted from km/h to m/s (36 / 3.6 = 10 m/s). Reaction distance = speed × reactionTimeSeconds = 10 m. Braking distance = speed² / (2×maximumDecelerationMps2) = 100 / 10 = 10 m. Total stopping distance = 20.0 m.

7. Emergency brake decision (shouldEmergencyBrake)

Every valid obstacle (not just the nearest) is tested against all three conditions: in front (forward ≥ 0), inside the lane corridor (|left| ≤ 1.50), and within stopping distance (forward ≤ 20.0). 'barrier' (forward=18.0, left=1.0) satisfies all three and triggers ENGAGE; 'cone' fails the lane-corridor test (|4.0| > 1.50) despite being nearest; 'crate' fails the stopping-distance test (25.0 > 20.0).

8. Output

main.cpp prints the count of valid obstacles, the nearest obstacle's id and world position, the stopping distance, and the final brake state, producing the exact expected output block.

Verified Build/Run Output

Rebuilt via cmake --build build --config Release and run via .\build\Release\rover_safety_monitor.exe:

```
ARL Rover Safety Monitor
Valid detections: 3
Nearest obstacle: cone
World position: (96.0, 62.0)
Stopping distance: 20.0 m
Emergency brake: ENGAGE
```

This matches the specification's required output exactly, and the same corrected logic (validity filtering, coordinate transform, nearest-obstacle selection, unit-consistent stopping-distance formula, and full-scan multi-condition braking check) generalizes correctly to other telemetry values, since no part of the fix was specific to this particular input — each correction addressed a general formula or comparison error, not the example's specific numbers.

References and Resources Used

- C++ reference documentation (cppreference.com) for std::optional, std::hypot, std::isfinite, and std::fabs semantics.
- Standard 2D rotation matrix / change-of-basis formulas for converting a local (forward, left) sensor frame into a world (X, Y) frame, verified numerically against the assignment's own worked example (cone obstacle, heading 90°) and cross-checked with an independent 60°-heading example to confirm both sine and cosine terms are needed at non-axis-aligned headings.
- Standard kinematics equation $v_{final}^2 = v_{initial}^2 - 2a \cdot d$, solved for d with $v_{final} = 0$, to derive the braking-distance term $d = v^2 / (2a)$ used in calculateStoppingDistance.
- Anthropic's Claude (Sonnet, in the Claude.ai chat interface) was used as an AI pair-debugging assistant throughout this assignment: to interpret CMake/VS Code build and IntelliSense error output, to identify and explain each logic/sign/unit bug listed above, to verify corrected formulas numerically against the specification's worked example, and to review manually-written fix attempts (including two iterations of shouldEmergencyBrake) for remaining errors before final build verification.